

Risk Management in a Financial Services Company

21CSC205P Database Management Systems
A MINI PROJECT REPORT

Submitted by

ADITYA VARDHAN SHARMA
[RA2411030010282]
AVOY DEBNATH [RA2411030010308]

Under the Guidance of

(Dr. Rajalakshmi A)

(Asst. Professor, NWC)

*in partial fulfillment of the requirements for the degree
of*

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in (CYBER SECURITY)



DEPARTMENT OF NETWORKING AND COMMUNICATIONS
SCHOOL OF COMPUTING
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026



**SRM INSTITUTE OF SCIENCE AND
TECHNOLOGY KATTANKULATHUR – 603 203
BONAFIDE CERTIFICATE**

Certified that Project report titled “**RISK MANAGEMENT IN A FINANCIAL SERVICES COMPANY**” is the bonafide work of “**ADITYA VARDHAN SHARMA [RA2411030010282], AVOY DEBNATH [RA2410030010308]**” who carried out the **21CSC205P Database Management Systems** mini project work under my supervision.

Dr. Rajalakshmi A
Assistant Professor
NWC

HEAD OF THE DEPARMENT
PROFESSOR AND HEAD
Department

ABSTRACT

The rapid digital transformation of financial services has significantly increased exposure to cyber, operational, and regulatory risks. This case study examines a mid-sized financial services company that experienced phishing attacks, unauthorized database access, and malware infiltration due to the absence of a formal risk management framework. Using the S.C.Q.C.A. (Situation–Complication–Question–Conclusion–Analysis) framework, the study illustrates how reliance on isolated technical controls—without structured governance, comprehensive risk assessment, or incident response planning—creates systemic vulnerabilities across the organization.

The analysis highlights the importance of adopting recognized frameworks such as ISO 27001 and the NIST Cybersecurity Framework to enable continuous risk identification, mitigation, monitoring, and response. Findings show that proactive and integrated risk management not only strengthens cybersecurity resilience and regulatory compliance but also helps restore customer confidence. Ultimately, the study concludes that a comprehensive, ongoing risk management approach is essential for protecting digital assets and ensuring sustainable growth in modern financial organizations.

TABLE OF CONTENTS

Chapter No.	TITLE	Page No.
	ABSTRACT	iii
	LIST OF FIGURES	vi
	LIST OF TABLES	vii
1	Problem understanding, Identification of Entity and Relationships, Construction of DB using ER Model for their project	
	1.1 INTRODUCTION	1
	1.2 MOTIVATION	1
	1.3 SCOPE	2
	1.4 PROBLEM STATEMENT	2
	1.5 PROJECT REQUIREMENTS	2
	1.6 IDENTIFICATION OF ENTITY AND RELATIONSHIPS	3
	1.7 CONSTRUCTION OF DB USING ER MODEL FOR THE PROJECT	4
2	Design of Relational Schemas, Creation of Database and Tables for their project	
	2.1 RELATIONAL SCHEMA FOR THE PROJECT	5
	2.2 DESCRIPTION OF TABLE	6
	2.3 CREATION OF DATABASE AND TABLES	9
	2.4 INSERTION OF TUPLES INTO THE TABLE-DML COMMANDS	11
3	Complex queries based on the concepts of constraints, sets, joins, views, PL/SQL Procedure, Functions, Triggers and Cursors	
	3.1 ADDING CONSTRAINTS AND QUERIES BASED ON CONSTRAINTS	14
	3.2 QUERIES BASED ON AGGREGATE FUNCTIONS	15
	3.3 COMPLEX QUERIES BASED ON SETS	16
	3.4 COMPLEX QUERIES BASED ON JOINS	17
	3.5 COMPLEX QUERIES BASED ON VIEWS	18
	3.6 COMPLEX QUERIES BASED ON PROCEDURES WITH EXCEPTION HANDLING	19
	3.7 COMPLEX QUERIES BASED ON FUNCTIONS WITH EXCEPTION HANDLING	20

	3.8 COMPLEX QUERIES BASED ON TRIGGERS	21
	3.9 COMPLEX QUERIES BASED ON CURSORS	22
	3.10 COMPLEX QUERIES BASED ON TRIGGERS	24
4	ANALYZING THE PITFALLS, IDENTIFYING THE DEPENDENCIES, AND APPLYING NORMALIZATIONS	
	4.1 ANALYSE THE PITFALLS IN RELATIONS	28
	4.2 FIRST NORMAL FORM	28
	4.3 SECOND NORMAL FORM	29
	4.4 THIRD NORMAL FORM	31
	4.5 BCNF	32
	4.6 FOURTH NORMAL FORM	33
	4.7 FIFTH NORMAL FORM	35
5	Implementation of concurrency control and recovery mechanisms.	
	5.1 INTRODUCTION TO TRANSACTIONS	37
	5.2 TRANSACTION CONTROL LANGUAGE (TCL)	37
	5.3 CREATE 5 TRANSACTIONS FOR YOUR PROJECT AND EXECUTE	38
	5.4 CONCURRENCY CONTROL	40
6	Front-end And Back-End Implementation code	
	6.1 FRONT-END MODULE CODES	43
	6.2 BACK-END MODULE AND DATABASE CONNECTIVITY	46
7	Results and Discussions	
	7.1 SCREENSHOTS OF FRONT-END	48
	7.2 SCREENSHOTS OF DATABASE	50

LIST OF FIGURES

Figure No.	Name	Page no.
1	ER Diagram of Database-Driven Vulnerability & Patch Management System	10
2	Relational Schema Diagram	11
3	Dash board Tab	48
4	Risk Management Tab	48
5	ADD Customer Tab	49
6	ADD Account Tab	49
7	ADD Logic risk Tab	50
8	Databases	50

LIST OF TABLES

Table No.	Table Name	Page no.
1	Customer Table Schema	12
2	Account Table Schema	12
3	Transaction Table Schema	13
4	Department Table Schema	13
5	Risk Table Schema	13
6	Risk_Assessment Table Schema	14
7	Mitigation_Action Table Schema	14

CHAPTER 1

PROBLEM UNDERSTANDING, IDENTIFICATION OF ENTITY AND RELATIONSHIPS, CONSTRUCTION OF DB USING ER MODEL FOR THE RISK MANAGEMENT IN A FINANCIAL SERVICES COMPANY

1.1 Introduction

Risk management is a critical function in financial services companies, as these organizations deal with large volumes of sensitive data, financial transactions, regulatory requirements, and market uncertainties. Effective risk management requires accurate data collection, structured storage, and reliable analysis.

A well-designed database system supports this process by organizing risk-related information such as customers, accounts, transactions, risk assessments, controls, and incidents. The Entity–Relationship (ER) model provides a conceptual framework to represent these real-world components and their interactions, forming the foundation for a robust database.

This project focuses on understanding the risk management domain, identifying key entities and relationships, and constructing a database using the ER model.

1.2 Motivation

The motivation for this project arises from the increasing complexity of financial transactions and regulatory requirements. Manual or poorly structured systems make it difficult to track risks effectively. Designing a database using the ER model improves data consistency, reduces redundancy, and enables financial institutions to respond quickly to potential risks.

1.3 Scope

The scope of this project includes:

- Understanding risk management processes in a financial services company
- Identifying core entities such as customers, accounts, risks, and transactions
- Defining relationships among these entities
- Designing an ER model
- Converting the ER model into a logical database structure

1.4 Problem Statement

Financial services companies face challenges in managing risk data due to fragmented systems, unstructured storage, and lack of standardized relationships among datasets.

Problem:

To design a structured database using the ER model that effectively captures risk-related information and supports monitoring, assessment, and reporting of risks within a financial services organization

1.5 Project Requirements

- Identification of different types of risks
- Storage of customer and transaction data
- Tracking risk assessments and risk levels
- Maintaining mitigation actions and responsible departments
- Ensuring data integrity and easy retrieval
- Use of ER modeling techniques for database design

1.6 Identification of Entity and Relationships

Entities identified:

- **Customer** – stores customer details
- **Account** – represents customer accounts
- **Transaction** – records financial transactions
- **Risk** – captures risk type and description
- **Risk Assessment** – evaluates risk severity and probability
- **Mitigation Action** – actions taken to reduce risk
- **Department** – responsible units handling risks

Relationships:

- A Customer owns one or more Accounts
- An Account performs many Transactions
- A Transaction may be associated with one or more Risks
- A Risk is evaluated through Risk Assessments
- A Risk is managed by a Department
- A Risk can have multiple Mitigation Actions

1.7 Construction of DB Using ER Model for the Risk Management in a Financial Services Company

The ER model is constructed by representing each entity as a rectangle, attributes as ovals, and relationships as diamonds. Primary keys uniquely identify records, while foreign keys establish relationships between entities. This ER-based database design ensures efficient organization of risk data, minimizes redundancy, and supports effective risk monitoring and decision-making in a financial services company.

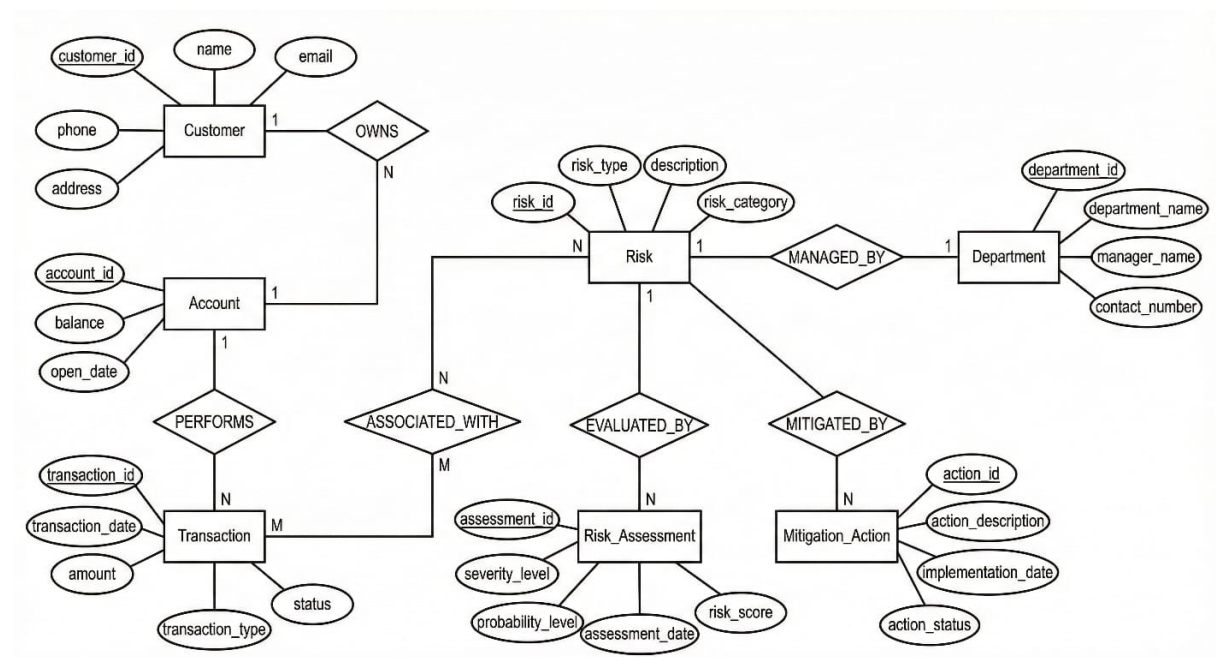


Figure 1.1: ER Diagram for Risk Management in a Financial Services Company

Chapter 2

Design of Relational Schemas, Creation of Database and Tables for Risk Management in a Financial Services Company

2.1 Relational Schema for Risk Management in a Financial Services Company

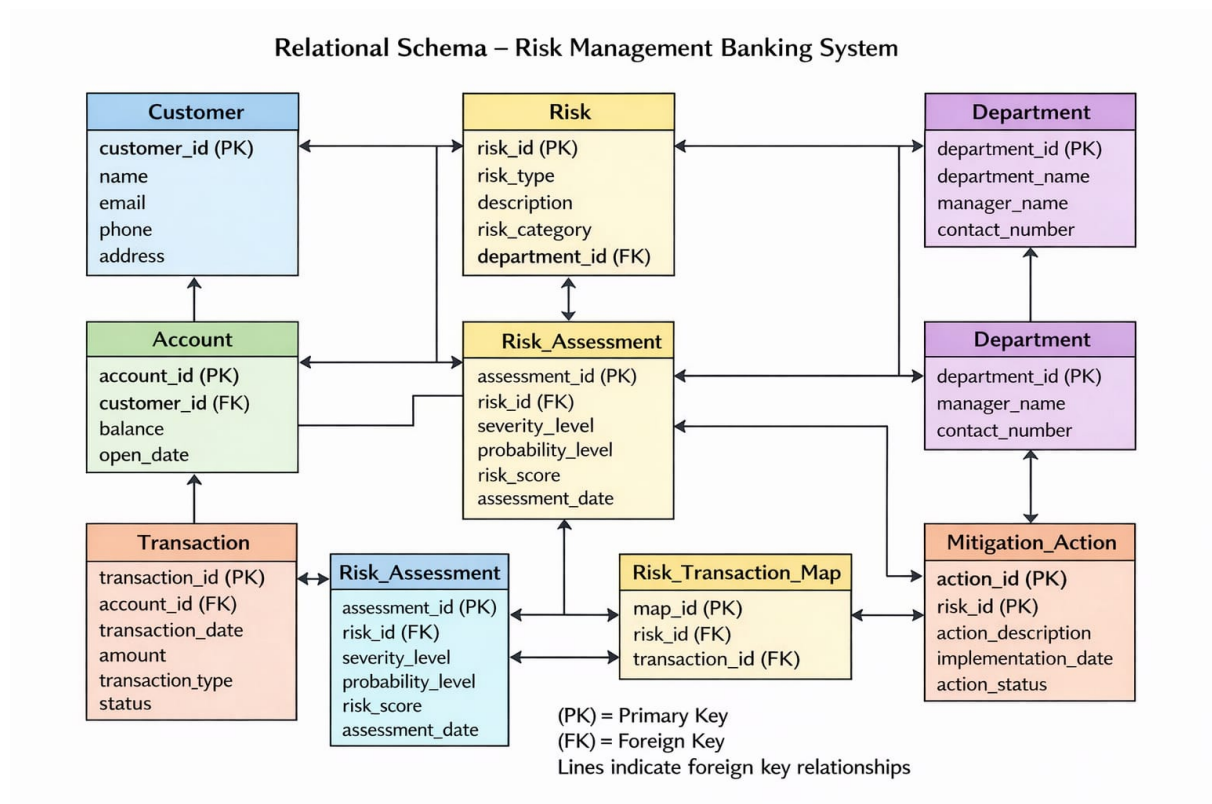


Figure 2.1: Relational Schema for Risk Management in a Financial Services Company

The figure 2.1 depicts the relational schema for the Risk Management in a Financial Services Company.

2.2 Description of Tables

2.2.1 CUSTOMER

N0.	Attribute Name	Data Type	Size	Constraint
1	customer_id	INT	-	Primary Key
2	customer_name	VARCHAR	100	NOT NULL
3	email	VARCHAR	100	UNIQUE
4	phone	VARCHAR	15	NOT NULL

2.2.2 ACCOUNT

N0.	Attribute Name	Data Type	Size	Constraint
1	account_id	INT	-	PRIMARY KEY
2	customer_id	INT	-	FOREIGN KEY
3	account_type	VARCHAR	30	NOT NULL
4	balance	DECIMAL	10,2	DEFAULT 0

2.2.3 TRANSACTION

N0.	Attribute Name	Data Type	Size	Constraint
1	transaction_id	INT	-	PRIMARY KEY
2	account_id	INT	-	FOREIGN KEY
3	amount	DECIMAL	10,2	NOT NULL
4	transaction_date	DATE	-	NOT NULL

2.2.4 DEPARTMENT

N0.	Attribute Name	Data Type	Size	Constraint
1	department_id	INT	-	PRIMARY KEY
2	department_name	VARCHAR	50	NOT NULL

2.2.5 RISK

N0.	Attribute Name	Data Type	Size	Constraint
1	risk_id	INT	-	PRIMARY KEY
2	transaction_id	INT	-	FOREIGN KEY
3	department_id	INT	-	FOREIGN KEY
4	risk_type	VARCHAR	50	NOT NULL
5	description	VARCHAR	200	-

2.2.6 RISK_ASSESSMENT

N0.	Attribute Name	Data Type	Size	Constraint
1	assessment_id	INT	-	PRIMARY KEY
2	risk_id	INT	-	FOREIGN KEY
3	risk_level	VARCHAR	20	NOT NULL
4	probability	INT	-	CHECK (probability BETWEEN 1 AND 5)

2.2.7 MITIGATION_ACTION

N0.	Attribute Name	Data Type	Size	Constraint
1	action_id	INT	-	PRIMARY KEY
2	risk_id	INT	-	FOREIGN KEY
3	action_description	VARCHAR	-	NOT NULL

2.3 Creation of Database and Tables - DDL Commands

This section includes SQL Data Definition Language (DDL) commands used to create the database and tables.

```
CREATE DATABASE RiskManagementDB;
```

```
USE RiskManagementDB;
```

```
CREATE TABLE Customer (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    phone VARCHAR(15) NOT NULL  
);
```

```
CREATE TABLE Account (  
    account_id INT PRIMARY KEY,  
    customer_id INT,  
    account_type VARCHAR(30) NOT NULL,  
    balance DECIMAL(10,2) DEFAULT 0,  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
);
```

```
CREATE TABLE Transaction (  
    transaction_id INT PRIMARY KEY,  
    account_id INT,
```



```
amount DECIMAL(10,2) NOT NULL,  
  
transaction_date DATE NOT NULL,  
  
FOREIGN KEY (account_id) REFERENCES Account(account_id)  
  
);
```

```
CREATE TABLE Department (  
  
department_id INT PRIMARY KEY,  
  
department_name VARCHAR(50) NOT NULL  
  
);
```

```
CREATE TABLE Risk (  
  
risk_id INT PRIMARY KEY,  
  
transaction_id INT,  
  
department_id INT,  
  
risk_type VARCHAR(50) NOT NULL,  
  
description VARCHAR(200),  
  
FOREIGN KEY (transaction_id) REFERENCES Transaction(transaction_id),  
  
FOREIGN KEY (department_id) REFERENCES Department(department_id)  
  
);
```

```
CREATE TABLE Risk_Assessment (  
  
assessment_id INT PRIMARY KEY,  
  
risk_id INT,  
  
risk_level VARCHAR(20) NOT NULL,
```

```

probability INT CHECK (probability BETWEEN 1 AND 5),
FOREIGN KEY (risk_id) REFERENCES Risk(risk_id)
);

```

```

CREATE TABLE Mitigation_Action (
    action_id INT PRIMARY KEY,
    risk_id INT,
    action_description VARCHAR(200) NOT NULL,
    FOREIGN KEY (risk_id) REFERENCES Risk(risk_id)
);

```

2.4 Insertion of tuples into the table – DML commands

2.4.1 Customer Table

CustomerID	Name	Email	Phone
1	John Doe	john@gmail.com	9876543210
2	Mary Smith	mary@gmail.com	9123456789

2.4.2 Account Table

AccountID	CustomerID	AccountType	Balance
101	1	Savings	5000
102	2	Current	8000

2.4.3 Transaction Table

TransactionID	AccountID	Amount	Date
1001	101	2000	2026-01-10
1002	102	3000	2026-01-12

2.4.4 Department Table

DeptID	DeptName
1	Credit Risk
2	Operational Risk

2.4.5 Risk Table

RiskID	TransactionID	DeptID	Category	Description
201	1001	1	Credit Risk	Late repayment
202	1002	2	Operational Risk	System failure

2.4.6 Risk_Assessment Table

AssessmentID	RiskID	Severity	ImpactScore
301	201	High	4
302	202	Medium	3

2.4.7 Mitigation_Action Table

ActionID	RiskID	ActionPlan
401	201	Send payment reminder
402	202	Upgrade system security

CHAPTER 3

3.1 Adding Constraints and queries based on constraints

Question 1:

Add CHECK constraint to ensure balance is non-negative.

SQL Statement:

```
ALTER TABLE account  
ADD CONSTRAINT chk_balance CHECK (balance >= 0);  
.
```

Question 2:

Add UNIQUE constraint on email.

SQL Statement:

```
ALTER TABLE customer  
ADD CONSTRAINT unique_email UNIQUE (email);  
.
```

Question 3:

Retrieve accounts with balance greater than 5000.

SQL Statement:

```
SELECT account_id, balance  
FROM account  
WHERE balance > 5000;
```

Output (Sample):

account_id	balance
102	8000
103	150000
105	12000

3.2 Queries based on Aggregate Functions**Question 1:**

Find total balance of all accounts.

SQL Statement:

```
SELECT SUM(balance) AS total_balance FROM account;
```

Question 2:

Find average transaction amount.

SQL Statement:

```
SELECT AVG(amount) AS avg_transaction FROM transaction;
```

Question 3:

Find maximum and minimum probability.

SQL Statement:

```
SELECT MAX(probability) AS max_prob, MIN(probability)
AS min_prob
FROM risk_assessment;
```

Output (Sample):

total_balance	avg_transaction	max_prob	min_prob
42500	3400	5	2

3.3 Complex queries based on Sets

Question 1:

Retrieve customers with savings accounts.

SQL Statement:

```
SELECT customer_id FROM account  
WHERE account_type = 'Savings';
```

Question 2:

Retrieve customers with transactions above 5000.

SQL Statement:

```
SELECT a.customer_id  
FROM account a  
JOIN transaction t ON a.account_id = t.account_id  
WHERE t.amount > 5000;
```

Question 3:

Retrieve customers using UNION.

SQL Statement:

```
SELECT customer_id FROM account WHERE
```

```
account_type='Savings'  
UNION  
SELECT a.customer_id  
FROM account a JOIN transaction t  
ON a.account_id=t.account_id  
WHERE t.amount>5000;
```

Output (Sample):

customer_id
1
2
3

3.4 Complex queries based on Subqueries

Question 1:

Find accounts above average balance.

SQL Statement:

```
SELECT * FROM account  
WHERE balance > (SELECT AVG(balance) FROM account);
```

Question 2:

Find customer with highest balance.

SQL Statement:

```
SELECT customer_id FROM account  
WHERE balance = (SELECT MAX(balance) FROM account);
```

Question 3:

Find transactions above average transaction amount.

SQL Statement:

```
SELECT * FROM transaction  
WHERE amount > (SELECT AVG(amount) FROM  
transaction);
```

Output (Sample):

account_id	balance
103	15000
105	12000

3.5 Complex queries based on Joins

Question 1:

Display customer and account details.

SQL Statement:

```
SELECT c.customer_name, a.account_type  
FROM customer c  
JOIN account a ON c.customer_id = a.customer_id;
```

Question 2:

Display customer and transaction details.

SQL Statement:

```
SELECT c.customer_name, t.amount
FROM customer c
JOIN account a ON c.customer_id=a.customer_id
JOIN transaction t ON a.account_id=t.account_id;
```

Question 3:

Display customer with risk level.

SQL Statement:

```
SELECT c.customer_name, ra.risk_level
FROM customer c
JOIN account a ON c.customer_id=a.customer_id
JOIN transaction t ON a.account_id=t.account_id
JOIN risk r ON t.transaction_id=r.transaction_id
JOIN risk_assessment ra ON r.risk_id=ra.risk_id;
```

Output (Sample):

customer_name	risk_level
John Doe	High
Mary Smith	Medium
Arjun Mehta	Critical
Sara Khan	Low
David Wilson	Medium

3.6 Complex queries based on views

Question 1:

Create view for high-risk records.

SQL Statement:

```
CREATE VIEW high_risk AS  
SELECT * FROM risk_assessment  
WHERE risk_level IN ('High','Critical');
```

Question 2:

Create view for customer balances.

SQL Statement:

```
CREATE VIEW customer_balance AS  
SELECT c.customer_name, a.balance  
FROM customer c JOIN account a  
ON c.customer_id=a.customer_id;
```

Question 3:

Retrieve data from view.

SQL Statement:

```
SELECT * FROM high_risk;
```

Output (Sample):

risk_id	risk_level
201	High
203	Critical

3.7 Complex queries based on Procedures with exception handling

Question 1:

Create procedure to fetch balance.

SQL Statement:

```
DELIMITER //  
CREATE PROCEDURE get_balance(IN cid INT)  
BEGIN  
SELECT balance FROM account WHERE customer_id=cid;  
END //  
DELIMITER ;
```

Question 2:

Procedure with exception handling.

SQL Statement:

```
DELIMITER //  
CREATE PROCEDURE safe_balance(IN cid INT)  
BEGIN  
DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
SELECT 'Error occurred';  
SELECT balance FROM account WHERE customer_id=cid;  
END //  
DELIMITER ;
```

Question 3:

Call procedure.

SQL Statement:

```
CALL safe_balance(3);
```

Output (Sample):

balance
15000

3.8 Complex queries based on Functions with exception handling

Question 1:

Create function for risk level.

SQL Statement:

```
CREATE FUNCTION risk_fn(p INT)  
RETURNS VARCHAR(20)
```

DETERMINISTIC

```
RETURN IF(p>=5,'Critical','Normal');
```

Question 2:

Call function.

SQL Statement:

```
SELECT risk_fn(5);
```

Question 3:

Use function with table.

SQL Statement:

```
SELECT probability, risk_fn(probability) AS risk_level  
FROM risk_assessment;
```

Output (Sample):

probability	risk_level
5	Critical
4	Normal
3	Normal

3.9 Complex queries based on Triggers

Question 1:

Create trigger to prevent negative transaction.

SQL Statement:

```
CREATE TRIGGER trg_check  
BEFORE INSERT ON transaction  
FOR EACH ROW  
IF NEW.amount < 0 THEN  
SIGNAL SQLSTATE '45000';  
END IF;
```

Question 2:

Create trigger for logging.

SQL Statement:

```
CREATE TRIGGER log_insert  
AFTER INSERT ON account  
FOR EACH ROW  
INSERT INTO department VALUES (99,'Log');
```

Question 3:

Test trigger.

SQL Statement:

```
INSERT INTO transaction VALUES (2000,101,-500,'2026-02-01');
```

Output (Sample):

Error Message
Negative Transaction not allowed

3.10 Complex queries based on Cursors

Question 1:

Declare cursor.

SQL Statement:

```
DECLARE cur CURSOR FOR SELECT customer_name FROM  
customer;
```

Question 2:

Fetch data from cursor.

SQL Statement:

```
FETCH cur INTO cname;
```


Question 3:

Complete cursor procedure.

SQL Statement:

```
DELIMITER //
CREATE PROCEDURE cursor_demo()
BEGIN
DECLARE done INT DEFAULT 0;
DECLARE cname VARCHAR(50);
DECLARE cur CURSOR FOR SELECT customer_name FROM
customer;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET
done=1;

OPEN cur;
read_loop: LOOP
FETCH cur INTO cname;
IF done THEN LEAVE read_loop;
END IF;
SELECT cname;
END LOOP;
CLOSE cur;
END //
DELIMITER ;
```

Output (Sample):

customer_name
John Doe
Mary Smith
Arjun Mehta
Sara Khan
David Wilson

CHAPTER 4

ANALYZING THE PITFALLS, IDENTIFYING THE DEPENDENCIES, AND APPLYING NORMALIZATIONS

4.1 Analyse the Pitfalls in Relations

- **Redundancy** → customer data repeated
- **Update anomaly** → change name in multiple rows
- **Insert anomaly** → can't add customer without transaction
- **Delete anomaly** → deleting transaction removes customer info

Unnormalized Table (Before Normalization)

Customer_id	Customer_name	Account_id	Account_type	balance	Transaction_id	amount	Dept_name	Risk-type
1	John Doe	101	Savings	4000	1001	2000	Credit Risk	Credit Risk
1	John Doe	101	Savings	4000	2000	-500	Credit Risk	Credit Risk

4.2 First Normal Form(1NF)

4.2.1 Identify Dependency

- **customer_id** → customer_name
- **account_id** → account_type, balance
- **transaction_id** → amount

4.2.2: Apply Normalization to 1NF

Before 1NF

customer_id	customer_name	account_id	accounts	transactions
1	John Doe	101	Savings, Current	(1001,2000), (2000,-500)
2	Mary Smith	102	Current	(1002,3000)

After Applying 1NF

Customer_id	Customer_name	Account_id	Account_type	Transaction_id	amount
1	John Doe	101	Savings	1001	2000
1	John Doe	101	Current	2000	-500
1	John Doe	102	Current	1002	3000

4.3 Second Normal Form (2NF)

4.3.1: Identify Dependency

Partial dependencies:

- **account_id** → account_type, balance
- **customer_id** → customer_name

4.3.2: Apply Normalization to 2NF

Before 2NF

Customer_id	Customer_name	Account_id	Account_type	Transaction_id	Amount
1	John Doe	101	Savings	1001	2000
1	John Doe	101	Savings	2000	-500
2	Mary Smith	102	Current	1002	3000

After Applying 2NF

Customer Table

Customer_id	Customer_Name
1	John Doe
2	Mary Smith

Account Table

Account_id	Customer_id	Account_type
101	1	Savings
102	2	Current

Transaction Table

Transaction_id	Account_id	amount
1001	102	2000
2000	102	-500
1002	102	3000

4.4 Third Normal Form (3NF)

4.4.1: Identify Dependency

Transitive dependency:- risk_id → department_id → department_name

4.4.2: Apply Normalization to 3NF

Before 3NF(2NF tables)

From 2NF, we had:

ACCOUNT + RISK (combined issue)

Risk_id	Transaction_id	Department_id	Department_name	Risk_type
201	1001	1	Credit Risk	Credit Risk
202	1002	2	Operational Risk	Operational Risk

After Applying 3NF

RISK Table

risk_id	transaction_id	department_id	risk_type
201	1001	1	Credit Risk
202	1002	2	Operational Risk

DEPARTMENT Table

department_id	department_name
1	Credit Risk
2	Operational Risk

4.5 BCNF

4.5.1: Identify Dependency

risk_type → department_id (possible violation)

4.5.2: Apply Normalization to BCNF

Before BCNF (3NF Table)

From 3NF, we had the **RISK table**:

risk_id	transaction_id	department_id	risk_type
201	1001	1	Credit Risk
202	1002	2	Operational Risk
203	1003	5	Cyber Risk
204	1004	4	Compliance Risk
205	1005	3	Liquidity Risk

After Applying BCNF

RISK Table (BCNF)

risk_id	transaction_id	risk_type
201	1001	Credit Risk
202	1002	Operational Risk

RISK_TYPE Table

risk_type	department_id
Credit Risk	1
Operational Risk	2

DEPARTMENT Table (unchanged)

department_id	department_name
1	Credit Risk
2	Operational Risk

4.6 Fourth Normal Form(4NF)

4.6.1: Identify Dependency

Multi-valued dependency:- customer → multiple accounts AND multiple transactions

4.6.2: Apply Normalization to 4NF

Before 4NF (BCNF Tables)

RISK

risk_id	transaction_id	risk_type
201	1001	Credit Risk
202	1002	Operational Risk
203	1003	Cyber Risk
204	1004	Compliance Risk
205	1005	Liquidity Risk

After Applying 4NF

CUSTOMER

customer_id	customer_name
1	John Doe
2	Mary Smith

ACCOUNT

account_id	customer_id	account_type	balance
101	1	Savings	4000
102	2	Current	8000

TRANSACTION

transaction_id	account_id	amount
1001	101	2000
1002	102	3000

4.7 Fifth Normal Form(5NF)

4.7.1: Identify Dependency

Join dependency:-Customer ↔ Account ↔ Transaction

4.7.2: Apply Normalization to 5NF

Before 5NF (From 4NF)

We already have separated tables:

- CUSTOMER
- ACCOUNT
- TRANSACTION

After Applying 5NF (Short Tables)

CUSTOMER

customer_id	customer_name
1	John Doe
2	Mary Smith

ACCOUNT

account_id	customer_id
101	1
102	2

TRANSACTION

transaction_id	account_id
1001	101
1002	102

IMPLEMENTATION OF CONCURRENCY CONTROL AND RECOVERY MECHANISMS

5.1 Introduction to Transactions

A transaction is a group of database operations executed as a single unit to ensure data consistency and integrity.

5.1.1 Properties

◆ Properties (ACID)

- **Atomicity** → Ensures all Operations in a transaction
- **Consistency** → Ensures database remains in a Valid state before & after transaction
- **Isolation** → Ensures No interference between concurrent transactions
- **Durability** → Ensure committed remain Permanent stored

5.1.2 States

- **Active** – Transaction is currently executing
- **Partially Committed** – Final statement has been executed
- **Committed** – Transaction completed successfully
- **Failed** – Transaction cannot continue
- **Aborted** – Transaction has been rolled back

5.2 Transaction Control Language (TCL)

5.2.1 Save point :- Temporary checkpoint

5.2.2 Commit :- Save permanently

5.2.3 Rollback :- Undo changes

5.3 Create 5 transactions for your project and execute

Sample 1: Insert Transaction and Update Account

1. START TRANSACTION;
2. INSERT INTO transaction VALUES (3000, 101, 2000, '2026-03-01');
3. SAVEPOINT sp_trans;
4. UPDATE account SET balance = balance + 2000 WHERE account_id = 101;
5. COMMIT;
6. SELECT * FROM account WHERE account_id = 101;
7. ROLLBACK TO sp_trans;
8. SELECT * FROM account WHERE account_id = 101;

Output for Sample 1: Transaction Insert & Update

Query: SELECT * FROM account WHERE account_id = 101;

Account_id	Balance
101	6000

Sample 2: Insert Customer Record

1. START TRANSACTION;
2. INSERT INTO customer VALUES (6, 'Ravi Kumar', 'ravi@gmail.com', '90000000001');
3. SAVEPOINT sp_customer;
4. COMMIT;
5. SELECT * FROM customer WHERE customer_id = 6;
6. ROLLBACK TO sp_customer;
7. SELECT * FROM customer WHERE customer_id = 6;

Output for Sample 2: Customer Insert

Query: SELECT * FROM customer WHERE customer_id = 6;

customer_id	customer_name	email	phone
6	Ravi Kumar	ravi@gmail.com	90000000001

Sample 3: Update Risk Assessment

1. START TRANSACTION;
2. UPDATE risk_assessment SET risk_level = 'High' WHERE risk_id = 202;
3. SAVEPOINT sp_risk;
4. ROLLBACK TO sp_risk;
5. COMMIT;
6. SELECT * FROM risk_assessment WHERE risk_id = 202;

Output for Sample 3: Risk Update (After Rollback)

Query: SELECT * FROM risk_assessment WHERE risk_id = 202;

risk_id	risk_level	probability
202	Medium	3

Sample 4: Insert Risk Record

1. START TRANSACTION;
2. INSERT INTO risk VALUES (206, 1002, 2, 'Operational Risk', 'Test Risk');
3. SAVEPOINT sp_risk_insert;
4. COMMIT;
5. SELECT * FROM risk WHERE risk_id = 206;
6. ROLLBACK TO sp_risk_insert;
7. SELECT * FROM risk WHERE risk_id = 206;

Output for Sample 4: Risk Insert

Query: SELECT * FROM risk WHERE risk_id = 206;

risk_id	transaction_id	department_id	risk_type
206	1002	2	Operational Risk

Sample 5: Delete Transaction with Rollback

1. START TRANSACTION;
2. SAVEPOINT sp_delete;
3. DELETE FROM transaction WHERE transaction_id = 1005;
4. ROLLBACK TO sp_delete;
5. COMMIT;
6. SELECT * FROM transaction WHERE transaction_id = 1005;

Output for Sample 5: Delete with Rollback

Query: SELECT * FROM transaction WHERE transaction_id = 1005;

transaction_id	account_id	amount
1005	105	1500

5.4 Concurrency control

Concurrency control in DBMS is the technique of managing simultaneous transactions in a multi-user environment to ensure data consistency, integrity, and isolation .

5.4.1 Concurrency control Algorithms

Locking commands

a. Row-Level Locking – SELECT ... FOR UPDATE

Locks only selected rows during transaction.

1. START TRANSACTION;
2. SELECT * FROM account WHERE account_id = 101 FOR UPDATE;
3. UPDATE account SET balance = balance - 500 WHERE account_id = 101;

Row-Level Locking Output

Query:

```
SELECT * FROM account WHERE account_id = 101 FOR UPDATE;
```

account_id	balance
101	4000

b. Table-Level Locking – LOCK TABLE

Lock Modes

Lock Mode	Description
ROW SHARE	Allows concurrent access; prevents exclusive locks
ROW EXCLUSIVE	Used for insert/update/delete
SHARE	Allows read but not modification
SHARE ROW EXCLUSIVE	More restrictive shared lock
EXCLUSIVE	Full table lock

c. COMMIT – Release All Locks

Example:

```
START TRANSACTION;
```

```
UPDATE account SET balance = balance - 500 WHERE account_id = 101;
```

```
COMMIT;
```

d. ROLLBACK – Undo Changes & Release Locks

Example:


```
START TRANSACTION;  
UPDATE account SET balance = balance - 500 WHERE account_id = 101;  
ROLLBACK;
```

5.4.2 Example

Scenario: Preventing Multiple Updates on Same Account

1. START TRANSACTION;
2. SELECT * FROM account WHERE account_id = 101 FOR UPDATE;
3. UPDATE account SET balance = balance - 500 WHERE account_id = 101;
4. COMMIT;

Final Output

Query:

```
SELECT * FROM account WHERE account_id = 101;
```

account_id	balance
101	3500

FRONT-END AND BACK-END CODE OF Risk Management in a Financial Services Company

6.1 Front-End Module codes

The front-end of the system is developed using HTML, CSS, and JavaScript. It provides an interactive interface for users to view and manage customer, account, transaction, and risk data.

◆ Technologies Used

HTML → structure of pages

CSS → styling

JavaScript → API calls and dynamic updates

MySQL → For databases

Python → Backend Code

Flask (Python) → Database connectivity

◆ Key Front-End Files

1. base.html

Common layout (navbar/sidebar)

Used by all pages

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>RiskShield</title>
```

```
<link rel="stylesheet" href="/static/style.css">
```

```
</head>
```

```

<body>
  <nav>Risk Management System</nav>
  <div>
    {% block content %} {% endblock %}
  </div>
</body>
</html>

```

2. index.html (Dashboard)

Displays summary:

total customers

total accounts

total risks

```

<script>
fetch('/api/dashboard')
.then(res => res.json())
.then(data => {
  console.log(data);
});
</script>

```

3. customers.html

Displays customer list

```

<script>
fetch('/api/customers')

```

```

.then(res => res.json())
.then(data => console.log(data));

```

</script>

4. accounts.html

Displays account details

5. risks.html

Shows risk analysis with:

risk level

probability

department

6. main.js

Handles all API calls

```
fetch('/api/risks')
```

```
.then(res => res.json())
```

```
.then(data => {
```

```
  console.log(data);
```

```
});
```

6.2 Database connectivity

The backend is developed using Flask (Python) and connected to a MySQL database.

1) Database Configuration

```
DB_CONFIG = {  
    "host": "localhost",  
    "user": "root",  
    "password": "your_password",  
    "database": "riskmanagementdb"  
}
```

2) Connection Function

```
def get_connection():  
    return mysql.connector.connect(**DB_CONFIG)
```

3)Example API (Customers)

```
@app.route("/api/customers")  
def get_customers():  
    conn = get_connection()  
    cursor = conn.cursor()  
    cursor.execute("SELECT * FROM customer")  
    return jsonify(rows_to_json(cursor))
```

4)Example API (Risk Analysis)

```
cursor.execute("""  
SELECT r.risk_id, r.risk_type,
```

```
ra.risk_level, ra.probability,
```

d.department_name

FROM risk r

JOIN risk_assessment ra ON r.risk_id = ra.risk_id

JOIN department d ON r.department_id = d.department_id

""")

5) Features of Backend

REST APIs using Flask

MySQL database connectivity

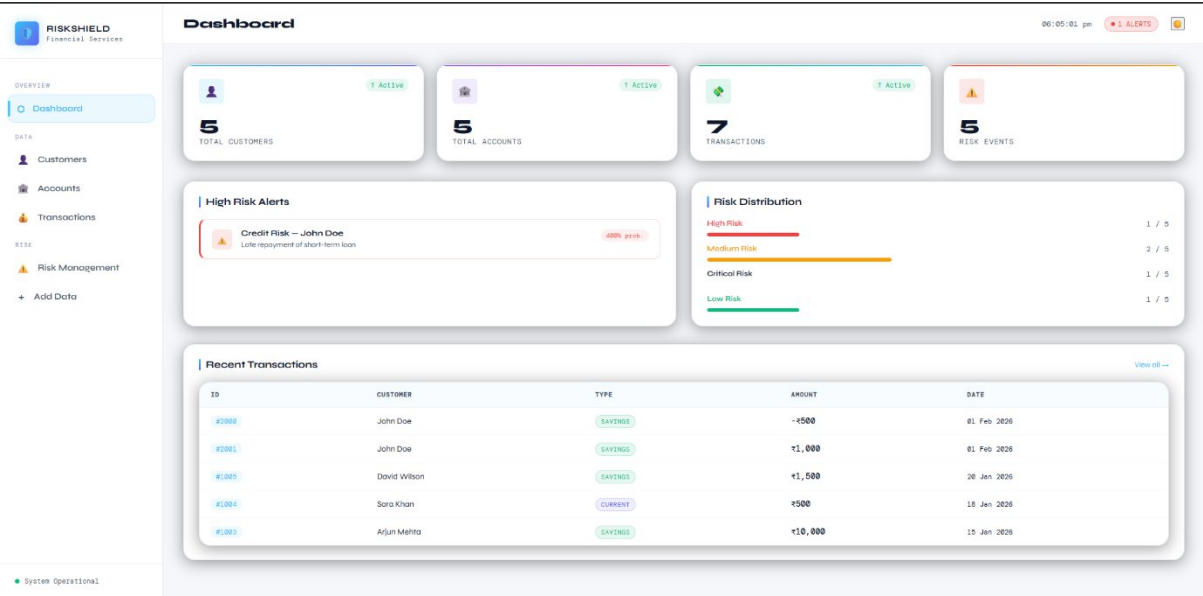
Multi-table joins

Data serialization (JSON)

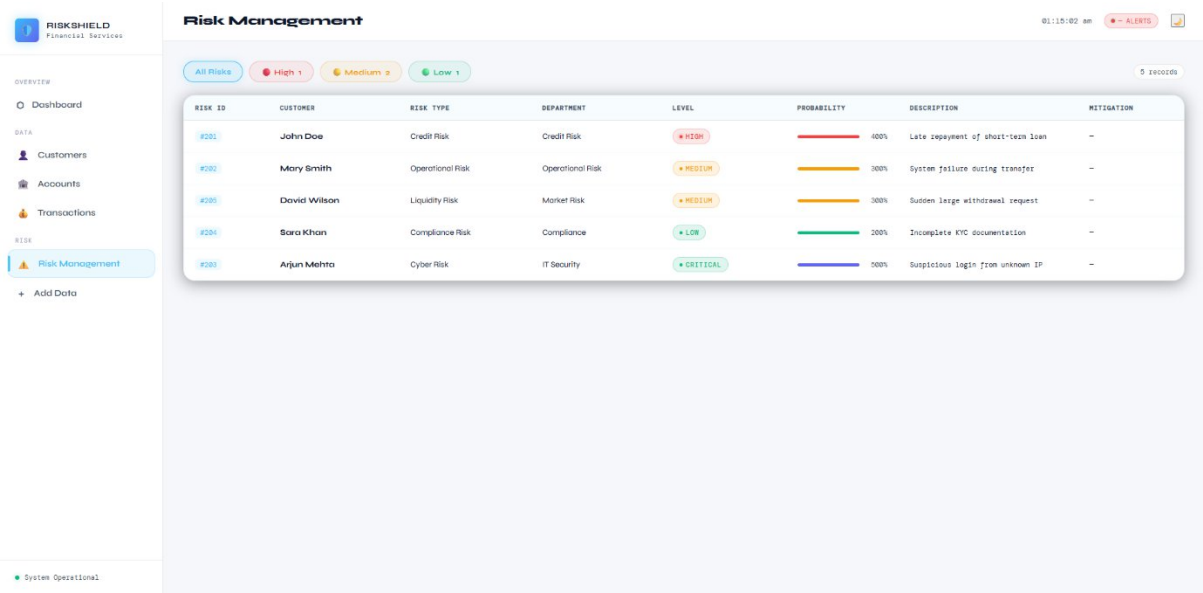
RESULTS AND DISCUSSIONS

7.1 Screenshots of front-end

7.1.1 Dashboard



7.1.2 Risk Management Tab



7.1.3 Add Customer Tab

OVERVIEW

Dashboard

DATA

Customers

Accounts

Transactions

RISK

Risk Management

+ Add Data

Add Data

Add Customer

Add Account

Log Risk

New Customer

Register a new customer in the system

FULL NAME

e.g. Arjun Sharma

EMAIL ADDRESS

arjun@example.com

PHONE NUMBER

+91 98765 43210

Add Customer →

System Operational

7.1.4 Add Account Tab

OVERVIEW

Dashboard

DATA

Customers

Accounts

Transactions

RISK

Risk Management

+ Add Data

Add Data

Add Customer

Add Account

Log Risk

New Account

Open a new account for an existing customer

SELECT CUSTOMER

Select Customer

ACCOUNT TYPE

Select Type

OPENING BALANCE (*)

0.00

Create Account →

System Operational

7.1.5 Add Logic Risk Tab

OVERVIEW

Dashboard

DATA

Customers

Accounts

Transactions

RISK

Risk Management

+ Add Data

Add Customer
Add Account

Log Risk

Log Risk Event

Record a new risk, assessment, and mitigation plan

TRANSACTION

DEPARTMENT

RISK TYPE

RISK LEVEL

PROBABILITY - 0%

RISK DESCRIPTION

MITIGATION ACTION

Log Risk

System Operational

7.2 Screenshots of Database

```

mysql> USE riskmanagementdb;
Database changed
mysql> show tables;
+-----+
| Tables_in_riskmanagementdb |
+-----+
| account                     |
| customer                    |
| customer_balance            |
| department                  |
| high_risk                   |
| normal_risk                 |
| risk                        |
| risk_assessment             |
| transaction                 |
+-----+
9 rows in set (0.03 sec)

mysql> Select * from customer;
+-----+-----+-----+-----+
| customer_id | customer_name | email                | phone |
+-----+-----+-----+-----+
| 1           | John Doe      | john@gmail.com       | 9876543210 |
| 2           | Mary Smith    | mary@gmail.com       | 9123456789 |
| 3           | Arjun Mehta   | arjun@outlook.com    | 9998776655 |
| 4           | Sara Khan     | sara@yahoo.com       | 9998989898 |
| 5           | David Wilson  | david.w@gmail.com    | 9765432109 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> Select * from account;
+-----+-----+-----+-----+
| account_id | customer_id | account_type | balance |
+-----+-----+-----+-----+
| 101        | 1           | Savings     | 4000.00 |
| 102        | 2           | Current     | 8000.00 |
| 103        | 3           | Savings     | 15000.00 |
| 104        | 4           | Current     | 2500.00 |
| 105        | 5           | Savings     | 12000.00 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> Select * from department;
+-----+-----+
| department_id | department_name |
+-----+-----+
| 1             | Credit Risk     |
| 2             | Operational Risk |
| 3             | Market Risk     |
| 4             | Compliance      |
| 5             | IT Security      |
+-----+-----+

```

```
MySQL 8.0 Command Line Client
6 rows in set (0.00 sec)

mysql> Select * from risk;
+-----+-----+-----+-----+-----+
| risk_id | transaction_id | department_id | risk_type | description |
+-----+-----+-----+-----+-----+
| 201 | 1001 | 1 | Credit Risk | Late repayment of short-term loan |
| 202 | 1002 | 2 | Operational Risk | System failure during transfer |
| 203 | 1003 | 5 | Cyber Risk | Suspicious login from unknown IP |
| 204 | 1004 | 4 | Compliance Risk | Incomplete KYC documentation |
| 205 | 1005 | 3 | Liquidity Risk | Sudden large withdrawal request |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> Select * from risk_assessment;
+-----+-----+-----+-----+
| assessment_id | risk_id | risk_level | probability |
+-----+-----+-----+-----+
| 301 | 201 | High | 4 |
| 302 | 202 | Medium | 3 |
| 303 | 203 | Critical | 5 |
| 304 | 204 | Low | 2 |
| 305 | 205 | Medium | 3 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> Select * from transaction;
+-----+-----+-----+-----+
| transaction_id | account_id | amount | transaction_date |
+-----+-----+-----+-----+
| 1001 | 101 | 2000.00 | 2026-01-10 |
| 1002 | 102 | 3000.00 | 2026-01-12 |
| 1003 | 103 | 10000.00 | 2026-01-15 |
| 1004 | 104 | 500.00 | 2026-01-18 |
| 1005 | 105 | 1500.00 | 2026-01-20 |
| 2000 | 101 | -500.00 | 2026-02-01 |
| 2001 | 101 | 1000.00 | 2026-02-01 |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

mysql>
```

References

Flask Documentation – <https://flask.palletsprojects.com>

MySQL Documentation – <https://dev.mysql.com/doc/>

W3Schools (HTML, CSS, JS) – <https://www.w3schools.com>

GeeksforGeeks – Database concepts and SQL queries